Allow programmer to control and detect coroutine elision by static constexpr bool should_elide() and coroutine_handle::elided()

Document #:	P2477R0
Date:	2021-10-14
Project:	Programming Language C++
Audience:	Evolution Working Group, Library Evolution Working Group

Reply-to:	Chuanqi Xu <chuanqi.xcq@alibaba-inc.com>

Contents

§ 1 Abstract

It is a well-known problem that coroutine needs dynamic allocation to work. Although there is an optimization in compiler, programmers couldn't control coroutine elision in a well-defined way. And there is not a good method that a programmer to use to detect whether the elision happened or not. So I proposed two methods. One for controlling the coroutine elision. And one for detecting it. Both of them wouldn't break any existing codes nor introduce any performance overhead.

§ 2 Background and Motivation

A coroutine needs space to store information (coroutine status) when it suspends. Generally, a coroutine would use `promise_type::operator new` (if existing) or `::operator new` to allocate coroutine status. However, it is expensive to allocate memory dynamically. Even we could use user-provided allocators, it is not easy (or very hard) to implement a safe and effective allocator. And dynamic allocation wouldn't be cheap than static allocation after all.

To mitigate the expenses, Richard Smith and Gor Nishanov designed HALO (which is called `coroutine elision` nowadays) to allocate coroutine on stack when the compiler find it is safe to do so. And there is a corresponding optimization called `CoroElide` in Clang/LLVM. But there is no word in the C++ specification about `coroutine elision`. It only says:

```
[dcl.fct.def.coroutine]p9
An implementation **may** need to allocate additional storage for a
        coroutine.
```

So it leaves the space for the compiler to do optimization. But it doesn't make any promise for programmers. So programmers couldn't find a well-defined method to control coroutine elision nor detecting whether the coroutine elision happened. And `coroutine elision` doesn't like other compiler optimization since programmers could feel strongly about the dynamic allocation. So I feel it is necessary to support it.

Except for the general need to avoid dynamic allocation to speed up. I heard from Niall Douglas that it is needed to disable `coroutine elision` in embedded system. In the resouces-limited system, we could use `promise_type::operator new` to allocate space from a big static-allocated bytes array. And the feature to disable `coroutine elision` is not present in the standard too.

§ 3 Terminology

This section aims at explaining terminology in this prposal to avoid misunderstanding. This aim is not to discuss the words used in the specification formally. That should be discussed in the wording stage.

§ 3.1 HALO and Coroutine elision

The two terminologies refer to the same thing in this proposal. I found that people on the language side perfer to use the term `HALO`. And the people on the compiler side prefer to use the term `Corotuine elision`. I like the term `Coroutine elision` more personally. We could discuss what's the prefer name later.

§ 3.2 constexpr time and compile time

Generally this two term refer to the same thing. But it's not the case for coroutine. For example, the layout of coroutine status is built at compile time. But the programmer couldn't see it. Informally, if a value is a constexpr-time constant, it could participate in the template/if-constexpr evaluation. And if a value is a compile-time constant, it needn't to be evaluated at runtime. So simply, a constexpr-time constant is a compile-time constant. But a compile-time constant may not be a constexpr-time constant.

§ 3.3 Coroutine status and Corotuine frame

Coroutine status is an abstract concept used in the language standard to refer all the information the coroutine should keep. And the coroutine frame refers to the data structure the compiler generated to keep the information needed. We could think coroutine frame is a derived class of coroutine status :).

§ 3.4 Ramp function

The compiler would split a coroutine function into several part: ramp function, resume function and destroy function. Both the resume function and destroy function contain a compiler generated state machine to make the coroutine work correctly. And the ramp function is the initial function which would initialize the coroutine frame.

§ 4 Design and Example

The design consists of 2 parts. One for controling the elision. One for detecting the elision. Both of them should be easy to understand.

§ 4.1 should_elide

If the compiler could find the name `should_elide` in the scope of promise_type, then if the results of `promise_type::should_elide()` evaluates to true at compile time, all the coroutine instance generated from the coroutine function are guaranteed to be elided. Or if the results of `promise_type::should_elide()` evaluates to false at compile time, all the coroutine instance generated from the coroutine function are guaranteed to not be elided. Otherwise, if the compiler couldn't find `should_elide` in the scope of promise_type or the results of `promise_type::should_elide` couldn't be evaluated at compile time, the compiler is free to elide every single coroutine instance generated from the coroutine function.

4.2 elided

Add a non-static member bool function `elided` to `std::coroutine_handle<>` and `std::coroutine_handle<PromiseType>`.

```
namespace std {
  template<>
  struct coroutine_handle<void>
  {
    // ...
    // [coroutine.handle.observers], observers
    constexpr explicit operator bool() const noexcept;
    bool done() const;
+   bool elided() const noexcept;
    // ...
  };

  template<class Promise>
  struct coroutine_handle
  {
    // ...
    // [coroutine.handle.observers], observers
    constexpr explicit operator bool() const noexcept;
    bool done() const;
+       bool elided() const noexcept;
        // ...
  };
}
```

And the semantic is clear. If the corresponding coroutine is elided, then `elided` would return true. And if the corresponding coroutine is not elided, then `elided` would return false.

4.3 Examples

A complete example could be find at: https://github.com/ChuanqiXu9/llvm-project/blob/CoroShouldElide/ShouldElide/ShouldElide.cpp

4.3.1 Always Enable/Disable coroutine elision

We could enable/disable coroutine elision for a kind of coroutine like:

```
struct TaskPromiseAlwaysElide : public TaskPromiseBase {
    static constexpr bool should_elide() {
        return true;
    }
};

struct TaskPromiseNeverElide : public TaskPromiseBase {
    static constexpr bool should_elide() {
        return false;
    }
};
```

Then for the coroutine like:

```
template<class PromiseType>
class Task : public TaskBase{
public:
    using promise_type = PromiseType;
    using HandleType = std::experimental::coroutine_handle<promise_type>;
    // ...
}; // end of Task
using AlwaysElideTask = Task<TaskPromiseAlwaysElide>;
using NeverElideTask = Task<TaskPromiseNeverElide>;

AlwaysElideTask always_elide_task () {
    // ... contain coroutine keywords
}
NeverElideTask never_elide_task () {
    // ... contain coroutine keywords
}
```

Then every coroutine instance generated from `always_elide_task()` would be elided. And every coroutine instance generated from `never_elide_task` wouldn't be elided.

§ 4.3.2 No guarantee to coroutine elision

If the compiler couldn't infer the result of `promise_type::should_elide` at compile time, then compiler is free to do elision or not. And the behavior would be the same with `should_elide` is not existed in `promise_type`.

```
bool undertermism;
struct TaskPromiseMeaningless : public TaskPromiseBase {
    static constexpr bool should_elide() {
        return undertermism;
    }
};
```

Then `TaskPromiseMeaningless` would be the same with `TaskPromiseBase` for every case.

§ 4.3.3 Controlling the elision at callsite

It is possible that we could control coroutine elision for specific call. Here is my demo,

```
using NormalTask = Task<TaskPromiseBase>;
using AlwaysElideTask = Task<TaskPromiseAlwaysElide>;
using NeverElideTask = Task<TaskPromiseNeverElide>;

struct ShouldElideTagT;
struct NoElideTagT;
struct MayElideTagT;

template<typename T>
concept ElideTag = (std::same_as<T, ShouldElideTagT> ||
                    std::same_as<T, NoElideTagT> ||
                    std::same_as<T, MayElideTagT>);

bool undetermism;
template <ElideTag Tag>
struct TaskPromiseAlternative : public TaskPromiseBase {
    static constexpr bool should_elide() {
        if constexpr (std::is_same_v<Tag, ShouldElideTagT>)
            return true;
        else if constexpr (std::is_same_v<Tag, NoElideTagT>)
            return false;
        else
            return undetermism;
    }
};

template <ElideTag Tag = MayElideTagT>
using AlternativeTask = Task<TaskPromiseAlternative<Tag>>;

template <ElideTag Tag>
AlternativeTask<Tag> alternative_task () { /* ... */ } // This is a
        coroutine

int foo() {
  auto t1 = alternative_task<ShouldElideTagT>();
  // Task::elided would call coroutine_handle::elided
  assert (t1.elided());
  auto t2 = alternative_task<NoElideTagT>();
  assert (!t1.elided());
  // The default case, which would be general for most cases
  alternative_task();
}
```

From the example in `foo`, we could find that we get the ability to control elision at callsite finally! Although it looks tedious to add `template <ElideTag Tag>` alone the way. But I believe the cases that we need to control wouldn't be too much. We only need to control some of coroutines in a project.

§ 5 Issue

A big issue I see now is that we couldn't elide a coroutine into a function in another translation unit. Let's see the example:

```
// CoroType.h
class PromiseType {
public:
  static constexpr bool should_elide() {
     return true;
  }
  // ...
};
class AlwaysElideTask {
  using promise_type = PromiseType;
  // ...
};
AlwaysElideTask may_be_a_coroutine();      // We couldn't see if this is a
        coroutine
                                           // from the signature.
// A.cpp
AlwaysElideTask may_be_a_coroutine() {     // It is a coroutine indeed.
  co_await std::suspend_always{};
  // ... Other works needed.
}
// B.cpp
AlwaysElideTask CoroA() {
  co_return co_await may_be_a_coroutine(); // Is it a coroutine?
}
```

The example above shows the issue directly. We couldn't elide a coroutine defined in another translation unit since we couldn't know if it is coroutine even!

Could we solve this problem by adding specicial function attribute to mark a function is a coroutine? No, we can't. Since at least we need to know how many bits we need to allocate the coroutine status if we want to do elide. And compiler couldn't infer that if it coulnd't see the body of the coroutine. The key point here is that the compiler would compile translation unit one by one and the compiler couldn't see the intents from other translation unit.

From the perspective of a compiler, this issue is potentially solvable by enabling LTO (Link Time Optimization). In LTO, the compiler could see every translation unit. But since we are talking about the language standard. I feel like that is is no possible that we could add the concept LTO in the standard nor we could change the compilation model.

So this problem is unsolvable if we didn't change the complation model. So the only solution I could image is that we constrain the semantics of `should_elide` by specifiying that we couldn't elide a coroutine in another translation unit explicitly in the standard.

§ 6 Implementation

A implementation could be found here: https://github.com/ChuanqiXu9/llvm-project/tree/CoroShouldElide

6.1 Current Implementation issue

An issue in the current implementation is that: it could work only if we turned optimization on. In other words, if we define this in the specification, it should work corrrectly under O0.

The reason is the internal pass ordering problem in the compiler. I definitely believe it could be solved if we get consensus on this proposal.

6.2 Implementation in GCC

Mathias Stearn points out that there is no coroutine elision optimization in GCC now. So it would be harder to implement this proposal in GCC since it would require GCC to implement coroutine elision first.

7 Q&A

This section includes question had been asked and the corresponding answer.

7.1 How the compiler guarantee coroutine elision if should_elide evaluates to true at constexpr time

Here talks about the case of clang only. Since the question raised because clang handles coroutine in two parts, the frontend part and the middle end part. But I think this wouldn't be a problem to GCC since GCC handles coroutine in the frontend.

Simply, the frontend would pass specified information to the middle end after parsing source codes. So when the frontend see the value of `promise_type::should_elide` evaluates to true or false, the frontend would pass specific marker to the middle end to inform that the coroutine should be elided or not.

Due to that coroutine elision depends on inlining now. So in the case middle end see a `always-elide` marker, it would mark the ramp function as `always_inline` to ensure it could be inlined as expected.

7.2 Is elided() a constexpr function?

No. Although elided() is evaluated at compile time, it is not evaluated at constexpr time. For example, the following function would print "elided is not constexpr function" all the way:

```
class Coro {
  Coro(coroutine_handle<> handle) : handle(handle) {}
```

```
  bool elided() {
    if constexpr(handle.elided())
      std::cout << "elided is constexpr function.\n";
    else
      std::cout << "elided is not constexpr function.\n";
  }
private:
  coroutine_handle<> handle;
};
```